

HiDiTingV100 低功耗软件

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档详细描述了 HiDiTingV100 的系统低功耗模式及应用开发指导，同时提供了常见问题的处理方法。用于指导用户低功耗开发及功耗调优。

产品版本

与本文档对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 概述.....1

2 特性配置.....3

3 深睡模式.....4

3.1 概述..... 4

3.2 睡眠流程..... 4

3.3 注意事项..... 5

4 浅睡模式.....6

4.1 概述..... 6

4.2 睡眠流程..... 6

4.3 注意事项..... 7

5 开发指引.....8

6 低功耗管理服务.....12

6.1 概述..... 12

6.2 接口说明..... 13

6.3 示例..... 14

7 低功耗调试.....16

7.1 功耗分解..... 16

7.2 典型场景功耗调试..... 16

7.2.1 睡眠功耗..... 17

7.2.2 运行功耗..... 18

7.2.3 蓝牙功耗..... 19

7.2.4 关机功耗.....	19
7.3 功耗 DFX.....	19
7.4 功耗优化原则	23
7.4.1 IO 通用配置.....	23
7.4.2 代码部署.....	25
7.4.3 外围器件功耗	25
8 常见问题.....	27
A 唤醒源.....	30
B GPIO 唤醒源说明	31
C 低功耗优化调试说明	32
D EVB 上 IO 调试方法	34

插图目录

图 1-2 睡眠状态 2

图 2-1 config.py 配置文件 3

图 3-1 深睡流程 5

图 4-1 浅睡流程 7

图 6-1 Power manager service 系统框架 13

表格目录

表 1-1 低功耗模式说明..... 1

表 5-1 低功耗接口说明..... 8

表 6-1 显示功耗管理服务成员函数..... 13

1 概述

低功耗是穿戴产品非常重要的一个特性，由于穿戴设备都是通过固定容量的电池供电，想要提升续航时间，只能通过降低典型场景平均电流来实现，其中最重要的场景就是深睡功耗值，同时动态运行功耗也是关键指标之一。BrandyK100 支持 2 种系统低功耗模式如表 1-1 所示，用户可根据实际应用场景选用对应的低功耗策略。

表1-1 低功耗模式说明

模式	芯片状态	特点
深睡模式	CPU 下电，RAM 不下电，AON 域以外其他外设下电，LDO 降压至 0.6V。 该模式仅支持 GPIO/RTC/UART/IPC 等唤醒源唤醒。	• CPU 进入掉电状态。 • 支持 tickless。 • 支持蓝牙 sniff 连接。 • 支持睡眠唤醒通知。 • 支持外设器件低功耗，唤醒后快速恢复。 • 支持模块可控下电。 • 支持多种唤醒源，快速对传感器响应。
浅睡模式	CPU 不下电，RAM 不下电，所有外设可控下电，LDO 不降压。 任意中断可退出浅睡模式。	• CPU 进入 WFI 状态。 • 支持 tickless。 • 支持蓝牙 sniff 连接。 • 支持所有外设正常工作，可控下电。 • 支持模块可控掉电。

模式	芯片状态	特点
		支持所有中断唤醒。

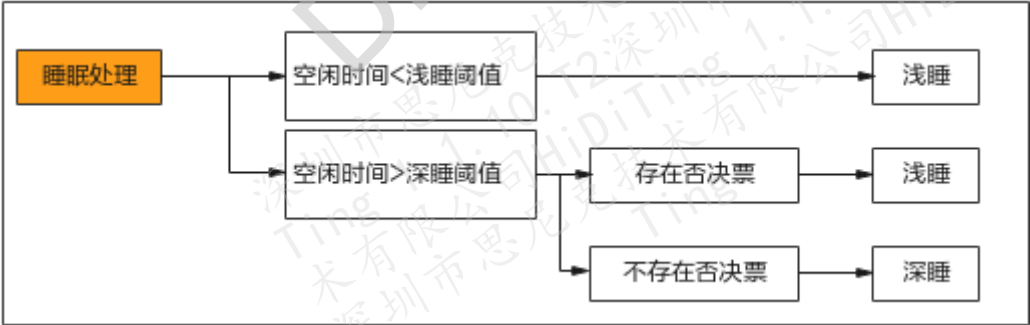
说明

- AON: Always ON 区域, 包括 RTC、GPIO、CWDT、Tsenssor、OTP、SSI/SWD 等。
- 浅睡模式和深睡模式互斥, tickless 和降压降频可根据业务场景与睡眠模式相结合。
- 默认提供的 target1 ~ target5 以及 target-js 版本在上电后就会进入低功耗, 执行业务 (测试 demo 或 Jlink 连接) 之前需要先发 AT 命令唤醒退出低功耗, 使用下面命令进行进出低功耗流程。
 - 低功耗唤醒, “AT^PMADDVOTE” 发送 2 次。
 - 业务验证完成后, 进入深睡, 发送一次 “AT^PMRMVOTE” 指令。

BrandyK100 的两种睡眠状态并不与具体的 RAM、外设的电压和频率相绑定, 而与系统是否空闲和用户投票相关。

低功耗模式开启后, 系统根据空闲时间和投票状态决定在空闲时进入深睡模式还是浅睡模式, 当前系统的浅睡阈值为 10ms, 深睡眠阈值为 20ms。如图 1-1 所示。

图1-2 睡眠状态



用户可结合应用场景主动调用调频接口来控制各类外设总线的开关和频率值。在 BrandyK100 中, 可以通过以下几种方式降低系统的功耗:

- Tickless: 通过移除空闲时间的 OS tick 计数的方式降低系统掉电次数及运行时间。
- 降压降频: 通过调压调频接口, 降低模块电压和频率, 减少不必要的损耗。针对外设器件, 非用即关。
- 浅睡模式: 系统进入浅睡模式, CPU 处于静止状态, 即 WFI。
- 深睡模式: 系统进入深睡模式, CPU 处于掉电状态, 依赖唤醒源唤醒。

2 特性配置

目前默认开启低功耗模式，使用“CONFIG_LOW_POWER_TEST”进行控制开启。用户需要关闭低功耗模式时，可以在编译版本前删除此宏。

具体设置编译宏方法如下：

步骤 1 在“software\code\sdk\build\config\target_config\brandy”目录下，打开“config.py”。

步骤 2 低功耗模式开关如图 2-1 所示，根据需要处理。

图2-1 config.py 配置文件

```
94 --- #target1: liteos + lvgl + nandflash + mipi + no psram + mini audio 典型单封装VGC版本
95 ✓ --- 'brandy-target1': {
96 ---     'board': 'evb',
97 ---     'base_target_name': 'target_standard_brandy_application_template',
98 ---     'std_libs': ['~:c++', '~:c++abi', '~:unwind'],
99 ✓ ---     'defines': ['PRE_ASIC', 'BRANDY_PRODUCT_EVB4', 'VERSION_STANDARD', 'LITEOS', 'CONFIG_OTA_UPDATE_SUPPORT',
100 ---                 'ALL_SOURCE', 'I2C_SLAVE_REG_ADDR_4BYTE', 'CONFIG_ZDIAG_NV_SUPPORT',
101 ---                 '~:TARGET_CHIP_BRANDY-1', '~:BRANDY_CHIP_V100-1', 'CFG_DRIVERS_NANDFLASH',
102 ---                 'CONFIG_ZDIAG_AUDIO_PROC_SUPPORT', 'CONFIG_ZDIAG_AUDIO_DUMP_SUPPORT', 'CONFIG_ZDIAG_AUDIO_PROBE_SUPPORT',
103 ---                 'SUPPORT_BLE', 'SUPPORT_BREDR', 'MEMORY_MINI', 'ENABLE_LVGL', 'SW_UART_DEBUG', 'SAVE_EXC_INFO',
104 ---                 'CONFIG_LOW_POWER_TEST', 'MIPI_ULPS_SUPPORT'],
105 ✓ ---     'ram_component': ['algorithm', 'dfx_port_brandy', 'dfx_update', 'dfx_nv', 'osal', 'arch_port', '~:testsuite',
106 ---                         'liteos_port', 'lcd', 'qspi_display',
107 ---                         'psram', 'hal_mipi', 'mipi_tx', 'non_os', 'touch',
108 ---                         'ulp_aon', 'hal_l2ram',
109 ---                         'x_dpai', 'x_vfs', 'x_disk', 'x_fat', 'drv_mmc', 'fs_yaffs2',
110 ---                         'pm_service', 'pm_brandy', 'cmn_header', 'at_cmd', 'graphic_at_service', 'app_at_service',
111 ---                         'audio_at_service', 'media_at_service', 'bt_manager_at_service',
112 ---                         'audio_proc', 'audio_dump', 'audio_probe'],
113 ---     'ram_component_set': ['bgh', 'media_mini', 'gpu', 'graphic_lvgl', 'dfx_set'],
114 ---     'image_analysis': True,
115 ---     'dsp_version': 'mini',
116 ---     'packet': True,
117 ---     'fs_image': False,
118 --- }
```

----结束

说明

目前所有 target 都支持并且默认开启低功耗功能。

3 深睡模式

3.1 概述

3.2 睡眠流程

3.3 注意事项

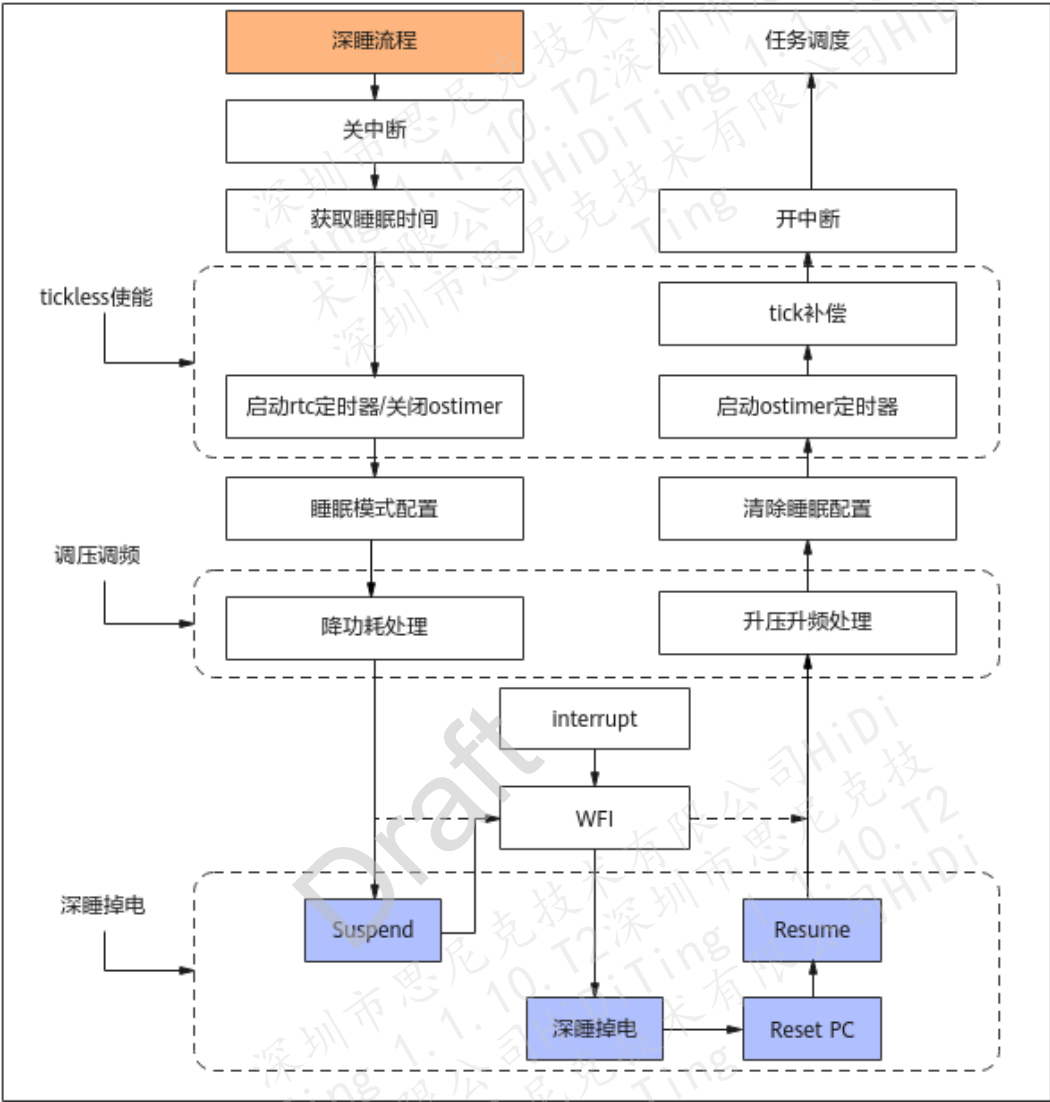
3.1 概述

BrandyK100 的低功耗策略是利用 IDLE 任务进行系统管理，由于 IDLE 任务优先级最低，因此该任务仅在系统空闲时得到执行，当低功耗管理系统判断没有即将执行的业务且不存在深睡否决票时，即可允许系统进入深睡模式，即 CPU、外设等下电，LDO 降压。该模式可以通过深睡唤醒源唤醒，深睡唤醒源包括 GPIO、IPC、Systick (RTC) 等，详细唤醒源可参见“A 唤醒源”。系统休眠时间采用 tickless 机制，避免每个 tick 都被唤醒。

3.2 睡眠流程

当系统状态满足深睡场景时，深睡眠场景的流程如图 3-1 所示。用户若在睡眠唤醒时处理关键业务，如保存数据、记录维测信息等，可通过注册回调函数的方式实现。系统睡眠前或唤醒后将执行用户注册的回调函数。

图3-1 深睡流程



说明

若深睡场景对 CPU、外设等掉电，则需在睡眠前后备份 (suspend) 和恢复 (resume) 现场。掉电场景唤醒后 CPU 从 Reset PC 开始运行，非掉电场景唤醒后则从 WFI 后继续执行。

3.3 注意事项

用户通过接口注册的回调函数，其中不建议执行耗时过长的操作或调用阻塞等待的接口。例如创建任务、等待信号量等操作，不应放在此类回调函数中。

4 浅睡模式

4.1 概述

4.2 睡眠流程

4.3 注意事项

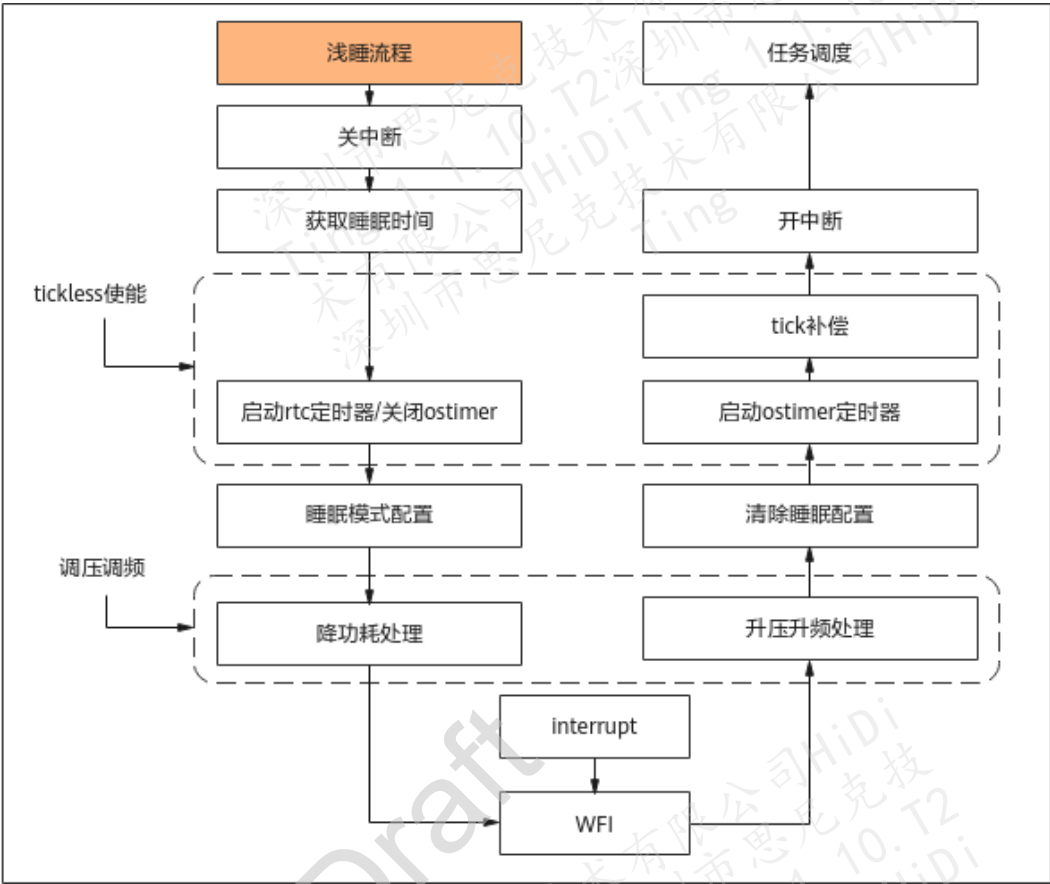
4.1 概述

浅睡模式与深睡模式流程类似，区别为 CPU 和外设不进行下电处理，任意中断触发将使系统退出浅睡模式。浅睡模式下 CPU 本身不会降低功耗，主要采用控制子模块功耗来降低功耗。如果需要深度定制（例如：开关外设时钟），需使能浅睡模式后注册接口，在入睡前和唤醒后对模块时钟进行开关操作。

4.2 睡眠流程

当系统状态满足浅睡场景时，浅睡眠场景的流程如图 4-1 所示。

图4-1 浅睡流程



4.3 注意事项

浅睡模式时间过短时，应取消 tickless，减少不必要的 tick 转换误差；浅睡模式时间过长时，应关闭不必要的外设及模块开关，同时应检查是否由于异常导致无法进入深睡模式。

5 开发指引

低功耗接口说明如表 5-1 所示。

表5-1 低功耗接口说明

接口名称	说明	文件路径
pm_get_sleep_info	获取睡眠相关信息，包括睡眠次数、睡眠时间等。	middleware/chips/brandy/pm/service/pm_sleep.c
pm_veto_get_info	获取否决票相关信息，包括投票 ID、投票历史等。	middleware/chips/brandy/pm/service/pm_veto.c
uapi_pm_init	设置低功耗参数，包括投票初始化、睡眠初始化等。	middleware/chips/brandy/pm/pm.c
uapi_sys_shipmode	芯片进入 shipmode 函数。	
uapi_pm_control	子系统上下电管理函数，包括 BT、DSP、DISPLAY 等子系统的上下电。	middleware/chips/brandy/pm/pm_adapt/pm_control.c
uapi_pm_request_mode	投票请求进入特定模式，包括 LP、NORMAL、PERFORMACE 等模式。	middleware/chips/brandy/pm/pm_adapt/pm_clocks.c
uapi_clocks_init	时钟初始化函数。	
pm_veto_init	投票初始化。	middleware/chips/brandy/pm/service/pm_veto.c
uapi_pm_add_sleep_veto	低功耗睡眠否决票，反对睡眠。	
uapi_pm_add_sleep_veto_with_timeout	带超时时间的低功耗睡眠投票。	

接口名称	说明	文件路径
uapi_pm_remove_sleep_veto	取消低功耗睡眠否决票，移除睡眠投票，允许睡眠。	
uapi_pm_register_fsm_handler	注册运行状态变化通知函数。	middleware/chips/brandy/pm/service/pm_fsm.c
uapi_pm_register_dev_ops	注册器件管理函数。	middleware/chips/brandy/pm/service/pm_dev.c
pm_unregister_dev_ops	取消注册器件管理函数。	

低功耗初始化流程示例如下：

步骤 1 系统低功耗初始化。

uapi_pm_init、uapi_clocks_init 和 pm_vote_init 需要在启动的过程中调用。

```
uapi_pm_init(); // 电源管理的初始化配置
pm_vote_init(); // 投票的初始化配置
uapi_clocks_init(); // 时钟的初始化配置
```

SDK 的发布包中已经调用了系统低功耗初始化，无需用户自己调用。

步骤 2 子系统上电。整个 BrandyK100 从软件上划分为 DISPLAY、DSP 和 BT 子系统，通过 uapi_pm_control 函数管理子系统的上下电。

```
uapi_pm_control(PM_DISPLAY_POWER_UP_AND_RUN); // DISPLAY子系统上电
uapi_pm_control(PM_DSP_POWER_UP_AND_RUN); // DSP子系统上电
uapi_pm_control(PM_BT_POWER_UP_AND_RUN); // BT子系统上电
```

下电的时候通过传不同的参数实现：

```
uapi_pm_control(PM_BT_POWER_OFF); // BT子系统下电
uapi_pm_control(PM_DSP_POWER_OFF); // DSP子系统下电
uapi_pm_control(PM_DISPLAY_POWER_OFF); // DISPLAY子系统下电
```

SDK 发布包中已经调用了子系统的上电，无需用户自己调用。

----结束

调试高低频和适配新器件示例如下：

- 模式和性能选择。用户可以根据自己的处理要求，自由选择模式，以获取更高的性能和更低的功耗。

不同的用户可以通过 `uapi_pm_request_mode` 去投票选择不同的模式，系统会依据优先满足性能的标准来决策出当前应该执行的模式。

```
errcode_t uapi_pm_request_mode(pm_id_t uid, pm_mode_t mode);

pm_id_t uid; // 用户投票的ID。

typedef enum {
    PM_MODE_NONE,
    PM_MODE_LP,
    PM_MODE_NORMAL,
    PM_MODE_PERFORMANCE,
    PM_MODE_MAX,
} pm_mode_t; // 可供选择模式
```

用户通过 `errcode_t` 的返回值来判断当前自己投票之后的决策结果。如下：

在 task1 中：

```
errcode_t ret1 = uapi_pm_request_mode(PM_ID_BSP, PM_MODE_NORMAL); // 系统进入PM_MODE_NORMAL模式
```

在 task2 中：

```
errcode_t ret2 = uapi_pm_request_mode(PM_ID_DEBUG, PM_MODE_PERFORMANCE); // 系统升级到PM_MODE_PERFORMANCE模式
```

当 task2 申请从 `PM_MODE_PERFORMANCE` 模式退出进 `LP` 模式：

```
ret2 = uapi_pm_request_mode(PM_ID_DEBUG, PM_MODE_LP); // 系统会决策，进PM_MODE_NORMAL模式，而不会进LP模式，保证两个task都能正常运行
```

用户可以通过 `ret2` 的返回值 `ERRCODE_PM_MODE_DOWNGRADED` 判断出虽然退出 `PM_MODE_PERFORMANCE`，但是没有进 `PM_MODE_LP` 模式。

- 注册器件进入深睡和退出深睡的处理函数。用户通过 SPI、I2C 等接口外挂器件注册器件进入深睡和退出深睡的处理函数。

```
errcode_t uapi_pm_register_dev_ops(pm_dev_id_t dev_id, pm_dev_ops_s dev_ops); // 注册器件睡眠唤醒回调函数

errcode_t pm_unregister_dev_ops(pm_dev_id_t dev_id); // 解注册器件睡眠唤醒回调函数

typedef struct {
```

```
uint32_t (*suspend)(void);    // 器件深睡回调函数原型

uint32_t (*resume)(void);     // 器件唤醒回调函数原型

uint32_t (*ioctl)(uint8_t type, uint8_t *data, uint32_t len); // 在低功耗中不使用，可忽略
} pm_dev_ops_t;
```

回调函数需要用户根据自己的器件手册和业务要求来进行处理，而且回调函数中不能使用信号量或者等中断等操作，因为在整个深睡和唤醒的流程中是不能够响应信号量或中断的。

- 投票。

uapi_pm_add_sleep_veto(PM_ID_SYS); // 当需要连续做业务或者持锁场景时，可以使用该接口。

uapi_pm_add_sleep_veto_with_timeout(PM_ID_SYS, 50); // 该接口也是持锁不睡眠，但是有定时器控制，不需要手动删除持锁。该定时器时间单位是ms。

uapi_pm_remove_sleep_veto(PM_ID_BSP); // 移除睡眠锁，和 uapi_pm_add_sleep_vote 必须成对使用。

只有当拥有投票权的所有 ID 都不持睡眠锁的时候，整系统才会进深睡。可以通过以下接口获取睡眠唤醒维测信息：

```
pm_get_sleep_info(); // 获取睡眠信息
```

```
pm_veto_get_info(); // 获取投票信息
```

6

低功耗管理服务

6.1 概述

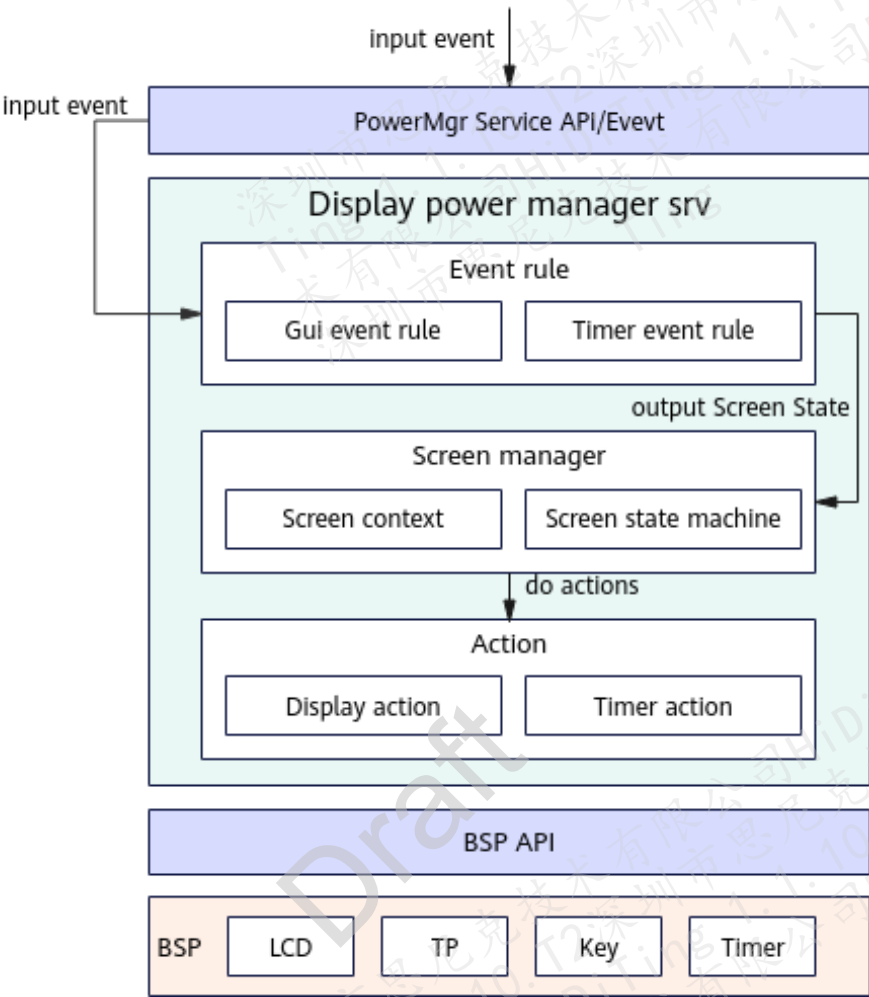
6.2 接口说明

6.3 示例

6.1 概述

低功耗管理服务，即 Power manager service 模块，当前用于屏幕的低功耗管理，主要实现亮屏和灭屏功能，向上对接上层 uikit/lvgl GUI 引擎及 native 应用，向下对接按键、触屏、显示等驱动模块，其服务框架如图 6-1 所示。

图6-1 Power manager service 系统框架



6.2 接口说明

Power manager service 提供统一对外接口 “power_display_svr_get_api” 用于获取显示功耗管理服务的 API 接口，其主要接口如表 6-1 所示。

表6-1 显示功耗管理服务成员函数

成员函数	说明	备注
turn_on_screen	控制屏幕进入亮屏状态。	-
turn_off_screen	控制屏幕进入黑屏状态。	-
get_screen_state	获取当前屏幕状态。	-

成员函数	说明	备注
set_screen_auto_off_timeout	设置屏幕超时灭屏时间。	- 配置前需要打开自动亮灭屏功能。 - 当前配置范围为[5s, 1h], 用户根据需要可进行二次开发。 - 若需要支持重启生效, 则需要另外进行保存 NV 项开发。
set_auto_timeout_function	设置自动超时灭屏功能是否生效。	-
set_screen_set_keep_on_timeout	应用设置一段时间保持亮屏。	当前配置保持亮屏时长最大为 1h, 即使配置超过 1h, 实际生效时长也为 1h; 小于等于 1h 按实际配置时长生效。用户可根据需要进行二次开发。
set_brightness	应用设置自定义亮度值。	-
get_brightness	获取应用设置屏幕亮度值。	-

6.3 示例

应用需要亮屏且保持一段时间不灭屏（当前点亮屏幕无操作一段时间会灭屏，默认 5s，可配置），设置方法如下：

步骤 1 获取显示功耗管理接口。

```
const power_display_svr_api_t *display_api = power_display_svr_get_api();
```

步骤 2 判断是否亮屏，如果为灭屏状态，则启动亮屏。

```
if (display_api->get_screen_state() != SCREEN_ON) {
    display_api->turn_on_screen();
}
```

步骤 3 控制 60s 保持亮屏。

```
display_api->set_screen_set_keepon_timeout(60000); // 入参单位为ms
```

如果在保持亮屏期间需要取消亮屏，恢复默认超时亮屏。（可选）

```
display_api->set_screen_set_keepon_timeout(0); // 入参单位为ms
```

----结束

须知

当前 pms 功能和 UI 线程任务合并，所以应用进入前设置长亮屏时间和应用退出后恢复长亮屏接口（配置 `display_api->set_screen_set_keepon_timeout(0);`）需要成对使用，否则如果连续调用长亮屏接口设置长亮屏时间，可能由于下发的配置在队列未生效场景，导致后续调用设置长亮屏时间的接口返回失败。

7

低功耗调试

本节主要针对低功耗的调试方法和优化手段进行说明，重点是对芯片功耗的调试和优化进行说明，并以 EVB 工程样机为例说明一般 IO 的配置方法，以及其它低功耗配置相关原则。

在调试低功耗前，首先需要了解硬件的各种电源管脚、硬件版图、管脚的 PINMUX（管脚多路复用）功能以及 PAD（芯片内部管脚）上的配置，这些内容请参考本芯片的《硬件用户指南》，本节重点是从软件方面介绍低功耗的优化。

7.1 功耗分解

7.2 典型场景功耗调试

7.3 功耗 DFX

7.4 功耗优化原则

7.1 功耗分解

通常所说的整机功耗，即电池上的功耗，为了便于调试和优化，我们将功耗分解为三个主要部分：主芯片的功耗、各类外设芯片的功耗、IO 管脚的功耗。其主芯片功耗和芯片主频、低功耗状态相关；各类外设芯片的功耗参考外设的器件手册；IO 功耗与板级硬件强相关，IO 功耗测量点位可以参考硬件原理图。本小节将对上述各类的配置一般原则进行说明，并基于典型场景给出功耗调试和优化的指导。

7.2 典型场景功耗调试

本芯片的典型功耗场景主要包括以下四类：

- 睡眠功耗：此场景即通常所说的待机状态，主芯片处于深睡状态（此睡眠指深睡状态），仅保留 XO32k 的时钟，其余时钟都被关闭，只能被 GPIO、RTC 等唤醒源进行唤醒，各类外设根据业务需求配置其状态（工作、睡眠或者下电），芯片不能工作，只能唤醒后继续处理任务。
- 运行功耗：此场景即通常所说的工作状态，主芯片的 CPU 处于工作态，所需的时钟都会正常打开，系统正常处理各种任务。
- 蓝牙功耗：此场景指蓝牙相关状态，芯片的蓝牙功耗指处理蓝牙协议和业务的 BT 核功耗，包含了典型的 BT sniff 保连场景、BLE 保连场景以及 BT 待机场景的功耗。
- 关机功耗：此场景指芯片处于 shipmode 模式的功耗，即芯片的最小工作单元运行时的功耗消耗。

7.2.1 睡眠功耗

睡眠功耗，重点关注本芯片的功耗消耗，如果在整机上测试需要关注 IO 漏电的功耗，此外外设的漏电也需要进行考虑。调试时参考以下步骤：

- 步骤 1 调试最小版本的功耗，最小版本只包含系统启动所需的基本单元。对本芯片来说，包含芯片功耗、NorFlash 和 PSRAM 的功耗，通常 NorFlash 和 PSRAM 的器件功耗从器件手册（实际应用中 PSRAM/NorFlash 供电，一般从板机供电，折算电流和板级选用 buck 还是 ldo 也有关系）中可知，此时测试的整机睡眠功耗再减去必要外设（例如 charger）的漏电，即可得到芯片的睡眠功耗，这个值也就是芯片的睡眠功耗。
- 步骤 2 调试 IO 的功耗，IO 功耗只和 IO 电平是否翻转有关，和芯片是否深睡无关。为便于调试，可在工作态调试 IO 漏电。芯片的管脚可以简单分为可唤醒的管脚和不可唤醒的管脚，具体参考“[B GPIO 唤醒源说明](#)”的说明。对于不可唤醒管脚，会默认在睡眠时关闭 IE 使能，唤醒后恢复初始状态；可唤醒管脚需要客户根据需要进行配置，对于 IO 的通常配置原则请参考“IO 通用配置”的说明。
- 步骤 3 调试外设的功耗，在完成上述的两项功耗调试后，可以逐步增加各类外设的功耗，并参考外设手册中的功耗值，将理论计算和实际测试中的功耗值进行比较，对于不符合预期的功耗值，进行逐步拆解和分析。对于较细致的功耗分解，也需要硬件设计时预留各类测试点，来分析功耗大的原因，一般优先预留主芯片的功耗测试点，各类 IO 电源的测试点，其次是各类外设的功耗测试点。可以参考本芯片《硬件用户指南》的“板级低功耗方案参考设计”章节内容。

----结束

7.2.2 运行功耗

运行功耗，重点关注芯片处于运行态下的功耗消耗，运行时的场景较多，本文不涉及外设和具体场景，只关注最小功能下的运行功耗，其它外设的新增功耗需要客户查阅对应的器件手册。调试时参考以下原则：

- 系统运行的动态功耗值，包含不同的档位，一般来说运行的频率越高，功耗值越大，实际使用中需要根据业务情况调档不同的频率，设置方法请参见“5 开发指引”中的模式和性能选择部分。基本原则是尽量让系统在低频运行，只在需要高频的业务下调高频，高频业务完成后，重新调成低频。
- 根据单板侧使用的不同 IO 管脚进行配置，详细配置请参见“IO 通用配置”。基本的原则关闭所有不用的管脚，使用的管脚根据睡眠状态和运行状态进行配置，使得 IO 漏电最小。例如 EVB 单板上 IO 漏电最优值约 5 μ A@1.8V。
- 运行功耗的其它优化手段。
 - 优化性能，减少运行时长，增大深睡的时间。
 - 减少唤醒次数，可以将实际的唤醒都打印，分析关闭无效或者低效的唤醒。
 - 关闭不使用模块的时钟，可进一步优化动态功耗，关闭方法如下：

代码路径：drivers/chips/brandy/clocks/drivers/acore/clocks_config_priv.c，关闭方法即将不使用的时钟配置为 TURN_OFF。

```
static const system_mcu_perip_clk_t g_system_mcu_perip_clk_set_config[] = {
    {HAL_CLOCKS_APP_SEC_CLK, TURN_ON},           // sec clock
    {HAL_CLOCKS_APP_DIAG_CLK, TURN_ON},          // diag clock
    {HAL_CLOCKS_APP_PWM_CLK, TURN_OFF},           // pwm clock
    {HAL_CLOCKS_APP_X2H_MEM_BUS_CLK, TURN_ON},    // x2h_mem_bus
clock

    /* Clocks M_CLKEN1 region */
    {HAL_CLOCKS_APP_TIMER_CLK, TURN_ON},          // timer clock
    {HAL_CLOCKS_APP_SPI0_M_CLK, TURN_OFF},         // spi0 clock
    {HAL_CLOCKS_APP_SPI1_MS_CLK, TURN_OFF},        // spi1 clock
    {HAL_CLOCKS_APP_SPI2_MS_CLK, TURN_OFF},        // spi2 clock
    {HAL_CLOCKS_APP_UART_H0_CLK, TURN_ON},         // uart h0 clock
    {HAL_CLOCKS_APP_UART_H1_CLK, TURN_ON},         // uart h1 clock
    {HAL_CLOCKS_APP_I2C0_CLK, TURN_ON},            // i2c0 clock
    {HAL_CLOCKS_APP_I2C1_CLK, TURN_ON},            // i2c1 clock
    {HAL_CLOCKS_APP_I2C2_CLK, TURN_OFF},           // i2c2 clock
    {HAL_CLOCKS_APP_I2C3_CLK, TURN_ON},            // i2c3 clock
    {HAL_CLOCKS_APP_QSPI_1CS_CLK, TURN_ON},        // qspi clock
    {HAL_CLOCKS_APP_XIP_BUS_CLK, TURN_ON},         // xip bus clock
}
```

```
{HAL_CLOCKS_APP_COM_BUS_DSP_CLK, TURN_ON},    // com bus dsp clock
{HAL_CLOCKS_APP_BT_TGTWS_CLK, TURN_ON},        // bt tgws clock  no
closed will affect bt call

/* Clocks M_CLKEN2 region */
{HAL_CLOCKS_APP_PULSE_CAPTURE_CLK, TURN_OFF}, //
pulse_capture_clken clock
{HAL_CLOCKS_APP_M_DAP_CLK, TURN_ON},           // m_dap_clken clock
{HAL_CLOCKS_APP_RC_CLK, TURN_ON},             // rc clock
{HAL_CLOCKS_APP_XO_CLK, TURN_ON},             // xo clock
};
```

7.2.3 蓝牙功耗

蓝牙功耗决定因素主要是蓝牙连接的 sniff 周期、单个周期内蓝牙的收发数据，以及蓝牙设备 A 和 B 之间的拉锯距离。

BT sniff 的蓝牙连接保连间隔典型值为 500ms；BLE 连接如第三方手机的无业务保连典型值为 240ms。BLE 的保连功耗值在相同条件下比 BT sniff 的值要小，但 BLE 保连的稳定性相对较差。

BLE 连接间隔通常由手机和手表协商确定，不同厂商手机 BLE 连接间隔差异较大，功耗也差别较大，SDK 支持手表侧进行主动协商，更新连接间隔。建议客户根据实际情况，使用合适的连接间隔，具体接口 gap_ble_connect_param_update，详细请参考《HiDiTingV100 BT/BLE 软件 开发指南》。

7.2.4 关机功耗

关机功耗对于整机来说包含芯片的功耗和充电器的功耗，此处主要说明芯片的 shipmode 功耗。芯片的 shipmode 模式指芯片只保留了最小区域 ULP 运行，仅内部 32kHz 时钟和 ULP GPIO 工作，支持关机闹铃唤醒、按键唤醒。一般应用在关机后保证芯片的最小供电。

实际使用中的约束：默认芯片 ULP 管脚处于三态，导致不同芯片的漏电不一致，因此在芯片完全掉电后首次上电且未启动前会有超预期的漏电，但只要启动一次后，软件会进行配置，此时再进入 shipmode 后漏电值就处于预期范围内。

7.3 功耗 DFX

功耗 DFX 维测用来分析和调试低功耗相关信息，主要包含以下内容：

- 打印信息分析，打印深睡时间和次数，以及上次是否进行过深睡。

```
APP|sys_slp: 0x1
```

```
APP|M core deepsleep time:1621, lightsleep time:0, m sleep flag:1
```

```
APP|M core deepsleep cnt:1, lightsleep cnt:0
```

```
APP|M core last veto id:1, last veto count:0
```

上述的打印中：

- 第一行“sys_slp”表示从上一次打印到本次打印的时间内整系统是否进入过深睡（1表示深睡过，0表示未深睡过）。
- 第二行“M core deepsleep time”表示MCU进入深睡的时间，“lightsleep”表示进入浅睡的时间，“m sleep flag”表示从上一次打印到本次打印的时间内MCU进入过深睡（1表示深睡过，0表示未深睡过）。
- 第三行是对应的睡眠次数。
- 第四行是最后一个投票的ID以及投票的次数。从中可以获取部分睡眠的信息。

- AT命令获取对应信息

- AT^SETPINPULL

此命令包含两个参数，设置示例如下：

```
[16:10:58.629]发→◇AT^SETPINPULL=129,2
```

```
[16:10:58.645]收←◆
```

```
OK~
```

“AT^SETPINPULL=x,x”中的参数说明如下：

- 参数一表示对应的pin序号，对应序号和PIN管脚的配置在文件中“drivers/chips/brandy/include/platform_core.h”中的enum pin_t的枚举中定义。
 - 参数二表示配置为配置的上下拉状态，0表示不配置，1表示下拉，2表示上拉。
 - 此命令配置重启后会丢失，不会保存。
- AT^GETPINPULL

示例如下：

```
[16:05:51.345]发→◇AT^GETPINPULL
```

```
[16:05:51.524]收←◆all pin lev:
```

```
i=0 level=0
```

```
i=1 level=1
```

```
i=2 level=0
```

```
i=3 level=0
```

```
i=4 level=0
i=5 level=0
i=6 level=0
i=7 level=1
i=8 level=1
```

此命令打印所有 PIN 管脚上下拉配置，由于管脚很多，上述只截取部分打印信息。示例中对应序号和所有 PIN 管脚的配置定义在代码中包含，文件位置为 “drivers/chips/brandy/include/platform_core.h”，其中 enum pin_t 枚举定义。

- AT^PMMODEGET

示例如下：

```
[16:05:02.171]发→AT^PMMODEGET
```

```
[16:05:02.303]收←◆APP|pm working mode is 1 at uid 10 vote.
```

```
APP|pm working ccrg clks[0x0] src 0x2, div 0x1.
```

```
APP|pm working ccrg clks[0x1] src 0x1, div 0x1.
```

```
APP|pm working ccrg clks[0x2] src 0x1, div 0x1.
```

```
APP|pm working ccrg clks[0x3] src 0x2, div 0x1.
```

```
APP|pm working ccrg clks[0x4] src 0x1, div 0x1.
```

```
APP|pm working ccrg clks[0x5] src 0x2, div 0x1.
```

```
APP|pm working ccrg clks[0x6] src 0x2, div 0x1.
```

```
APP|pm working ccrg clks[0x7] src 0x2, div 0x1.
```

```
APP|pm working ccrg clks[0x8] src 0x2, div 0x1.
```

```
APP|pm working ccrg clks[0x9] src 0x2, div 0x1.
```

```
APP|pm working ccrg clks[0xa] src 0x4, div 0x6.
```

```
APP|pm working ccrg clks[0xb] src 0x4, div 0x24.
```

```
APP|pm working ccrg clks[0xc] src 0x4, div 0x1.
```

```
APP|pm working ccrg clks[0xd] src 0x4, div 0x1.
```

```
APP|pm working ccrg clks[0xe] src 0x4, div 0x1.
```

```
last_veto_id=1, last_veto_lr=7044b13a, veto_timeout_timestamp=0, total_counts=1
```

```
sub_counts[0] = 0
```

```
sub_counts[1] = 0
```

```
sub_counts[2] = 0
```

```
sub_counts[3] = 0
```

```
sub_counts[4] = 0
```

```
sub_counts[5] = 0
```

```
sub_counts[6] = 0
```

```
sub_counts[7] = 0
```

```
sub_counts[8] = 0
```

```
sub_counts[9] = 0
```

```
sub_counts[10] = 1
```

```
sub_counts[11] = 0
```

```
sub_counts[12] = 0
sub_counts[13] = 0
sub_counts[14] = 0
sub_counts[15] = 0
g_lpc_mcpu_i2c_sleep_vetos=0
```

OK~

此命令获取当前的工作模式：性能模式、正常模式、低功耗模式。每个模式的电压和频率配置不同。上述“pm working ccrg”表示对应时钟的时钟源配置和分频系数配置，可对应代码进行问题分析；“sub_counts”对应的是进行投票禁止睡眠的 ID 信息操作，对应 ID 的定义在代码

“middleware/chips/brandy/pm/include/pm_definition.h”中，其定义如下：

```
typedef enum {
    PM_ID_BSP,
    PM_ID_SYS,
    PM_ID_BTH,
    PM_ID_BTC,
    PM_ID_AUDIO,
    PM_ID_VIDEO,
    PM_ID_DMA,
    PM_ID_DPU,
    PM_ID_VAU,
    PM_ID_BUTTON,
    PM_ID_DEBUG,
    PM_ID_ADC,
    PM_ID_DFX,
    PM_ID_SCREEN,
    PM_ID_STORAGE,
    PM_ID_TOUCH,
    PM_ID_MAX,
} pm_id_t;
```

工作模式定义如下：

```
typedef enum {
    PM_MODE_NONE,
    PM_MODE_LP,
    PM_MODE_NORMAL,
    PM_MODE_PERFORMANCE,
    PM_MODE_RESET,
    PM_MODE_MAX,
} pm_mode_t;
```

ID 值从 BSP 的 0 值开始，依次加 1 到最大值 TOUCH 的 15，表示每个模块持有的禁止深睡锁的数目；“g_lpc_mcpu_i2c_sleep_vetos”表示 I2C 持有禁止睡眠的锁数目。

7.4 功耗优化原则

系统需要保证稳定性、性能、低功耗的平衡，并且上述优先级从大到小，因此首先保证系统的功能和可用性，在此前提下，从多方面调优功耗，提高系统的续航能力。通用功耗调优的原则如下：

- 提高深睡占空比，即在系统完成业务工作后就进入深睡状态，系统中尽量不让任务在死循环中工作。
- 系统的能效比最优，即在系统需要高频时，完成工作后也马上切入到低频中，系统中尽量用合适的频率做合适的工作。
- 尽可能减少系统的唤醒源和外设的使用，减少唤醒源会让系统保持深睡状态，对功耗的优化有较大贡献；关闭不常用的外设，或者使用时打开，使用后关闭。
- 代码部署以及算法的优化，尽量让代码在“局部”运行，减少代码的来回跳转，同时根据需要把运算频繁和性能要求高的代码放置到运行速率高的内存上，也可优化功耗。

7.4.1 IO 通用配置

IO 配置基本原则：防止连接 IO 两端的芯片侧和外设器件侧的电压存在压差，导致出现漏电问题。本芯片中的 IO 配置包含以下选项，具体代码的配置选项路径为“drivers/chips/brandy/porting/include/pinctrl_porting.h”，配置结构体如下：

```
typedef struct {
    hal_pio_func_t func;
    hal_pio_drive_t drive;
    hal_pio_pull_t pull;
#ifdef CONFIG_PINCTRL_SUPPORT_IE
    hal_pio_ie_t ie;
#endif
} hal_pio_config_t;
```

各参数含义如下：

- “func”表示此管脚可配置的 pinmux 功能，例如可配置为 GPIO、I2C、UART 等。

- “drive” 表示此管脚的驱动能力，可配置的有四挡驱动能力，HAL_PIO_DRIVE_0 档驱动能力最弱，HAL_PIO_DRIVE_3 档驱动能力最强，HAL_PIO_DRIVE_MAX 表示不配置，采用芯片的默认值。
- pull 表示此管脚的弱上拉或者弱下拉配置，上下拉的电阻阻值约为 100kΩ，HAL_PIO_PULL_DOWN 表示下拉，HAL_PIO_PULL_UP 表示上拉，HAL_PIO_PULL_MAX 表示不进行上下拉的配置。
- “ie” 表示此管脚的输入使能是否打开，enable 表示打开，disable 表示关闭。

所有管脚的配置都放到一张表中，方便配置和修改，代码所在路径为

“drivers/boards/brandy_evb/board_config/board_evb.h”，表的 SDK 配置示例如下：

```
const hal_pio_config_t g_pio_function_config[PIN_MAX_NUMBER] = {
    { HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
      HAL_PIO_IE_ENBALE },    // S_EGPIO0
    { HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
      HAL_PIO_IE_ENBALE },    // S_EGPIO1
    { HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
      HAL_PIO_IE_ENBALE },    // S_EGPIO2
    { HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
      HAL_PIO_IE_ENBALE },    // S_EGPIO3
    { HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
      HAL_PIO_IE_DISABLE },    // S_EGPIO4 clk
    { HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
      HAL_PIO_IE_DISABLE },    // S_EGPIO5 cs
    { HAL_PIO_FUNC_OPI, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
      HAL_PIO_IE_ENBALE },     // S_EGPIO6
    { HAL_PIO_FUNC_OPI, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
      HAL_PIO_IE_ENBALE },     // S_EGPIO7
}
```

建议客户实际开发时根据硬件的设计，全部重新规划上述表格的配置。SDK 只提供了基于 EVB 板的最优功耗 IO 配置。

结合上述 IO 的配置方法，本芯片具体配置参考以下原则：

- 对无外设连接的管脚，IE 配置为 disable，PAD 配置为下拉。
- 对有外设连接的管脚，首先考虑深睡模式下的配置，保证两边电平配置一致，使用芯片的 PAD 的上拉或者下拉进行配置，其次考虑工作时的状态进行配置。
- 对只有输出的管脚，IE 可配置为 disable。
- 对板侧有上拉电阻或者下拉电阻的配置，需要结合 PAD 进行配置，防止出现电平不一致导致的漏电。

- 对部分深睡时下电的管脚，需要根据对端器件的状态使用 PAD 的上拉或者下拉进行配置，具体内容请参见“[B GPIO 唤醒源说明](#)”。
- 除以上基本原则，实际的管脚配置取决于实测 IO 在芯片深睡的漏电，选择最优值进行配置，其中对于 NorFlash、PSRAM、NandFlash 的管脚是固定的，SDK 中的配置已为最优配置，用户无需修改。

对于上述的管脚配置原则，结合 EVB 单板，对 SDK 的部分配置进行说明，详细内容请参见“[C 低功耗优化调试说明](#)”。

7.4.2 代码部署

代码的部署原则为尽量在局部运行，提高速率和性能，同时对不同的内存区域，代码的执行速率和性能也有差异，本芯片的内存类型和使用建议如下：

- ITCM：用来存放代码段的内存，执行速率最高，建议存放频繁运行的代码段，以及对性能要求特别高的代码段。
- DTCM：用来存放数据段的内存，执行速率最高，建议存放频繁运行的数据段，以及对性能要求特别高的数据段。
- L2RAM：用来存放数据或者代码的内存，执行速率低于 TCM，默认打开 CaChe 在某些场景下执行速率与 TCM 相当，可根据需要存放。
- PSRAM：属于外挂于芯片外内存，执行速率低于 L2RAM，默认打开 CaChe 在某些场景下执行速率与 TCM 相当，可根据需要存放。
- FLASH XIP：属于外挂于芯片外镜像存储，不仅可存放代码，同时也可在 Flash 上执行代码，并在进入 XIP 后可在其映射的内存空间上执行代码，执行速率最低，根据需要存放。

从上到下，内存的速率从高到低，建议用户将频繁调用或者性能要求高的代码根据需要进行存放，可修改链接脚本，其存放路径为

“drivers/boards/brandy_evb/linker/standard/acore.prelds”，具体修改方法请参考 SDK 中示例和链接脚本的通用规范。

7.4.3 外围器件功耗

外围器件的功耗，通常在器件手册中会有体现，本小节说明与芯片相关性较大的 NorFlash、PSRAM 和 NandFlash 的外设功耗。

- NorFlash 是本芯片需要外挂的版本存储器件，并支持 XIP 的方式运行代码。NorFlash 的 deep power down 模式功耗和运行功耗可以查看对应的器件手册，芯片的 SDK 会在进入深睡时，默认给 NorFlash 发送睡眠指令（通用指令为 0xB9

进入深睡，实际根据厂商手册确定），唤醒后会发送唤醒指令（0xAB 从深睡中恢复指令，实际根据厂商手册确定）。SDK 已支持多款主流 flash 的适配，通常的 flash 芯片 deep power down 模式功耗电流典型值为几 μA 级别。

- PSRAM (Pseudo Static Random Access Memory)，指代伪静态随机存储器。本芯片的 PSRAM 和芯片是合封在一起的，为进一步降低功耗，在深睡时通过发送指令将 PSRAM 进入 half-sleep 的状态，在唤醒后会退出 half-sleep 的状态。典型的功耗值（温度为 25°C ）为深睡时 $40\mu\text{A}@1.8\text{V}$ ，standby 功耗值为 $100\mu\text{A}@1.8\text{V}$
- NandFlash 是通过 SPI 或者 QSPI 的方式与芯片连接的，同 NorFlash 的连接协议相同。SDK 中使用“D EVB 上 IO 调试方法”中的方法获取一个最优功耗配置。同时为了进一步优化功耗，可以查阅器件手册，对于厂商支持睡眠模式，可在深睡和唤醒过程中转换。EVB 中配套的 NandFlash 默认不支持睡眠模式，因此代码中无转换模式的功能，EVB 上使用的器件 standby 下为功耗值为 $10 \sim 50 \mu\text{A}$ 。
- 其它外设的功耗可参考对应的器件手册，并可根据需求深睡时降低外设的功耗或者关闭外设。例如对于 I2C 连接的外设，通常在板级都设置有上拉电阻，因此在进入深睡后就需要对该上拉电阻的电源进行特殊配置，保证不会出现漏电问题，最终的配置为若上拉电阻的电源关闭，则 I2C 管脚配置为下拉；若上拉电阻的电源未关闭，则 I2C 管脚配置为上拉。

8 常见问题

1. 不能进入休眠模式，可能是什么原因？

不能进入休眠模式原因比较多，常见原因如下：

- 定时器使用错误（定时器时间小于睡眠阈值）
 - 举例：启动 10ms 周期定时器。
 - 分析：系统在入睡前会检查是否有定时器即将到期，如果定时器即将到期，系统则不允许进入系统低功耗模式。
 - 建议：根据实际业务启动定时器，在业务空闲时关闭对应定时器，特别是周期性定时器。
- 任务使用错误
 - 举例：任务体中循环操作，无主动释放动作，例如调用阻塞接口或 `uapi_sleep` 接口。
 - 分析：某一任务无主动释放动作，则其他低优先级任务得不到执行，严重时会引起看门狗复位。系统在入睡前会检查是否有任务即将被调度，如果存在任务主动释放过少，则系统不允许进入系统低功耗模式。
 - 建议：业务设计上尽量调用非阻塞接口，超时时间不应设置为无穷大而是合理超时时间。
- `osal_msleep` 使用错误
 - 举例：调用 `osal_msleep` 接口，传参为 10ms。
 - 分析：该行为效果类似于启动 10ms 周期定时器，该任务会以 10ms 为周期被调度，入睡前检查有任务即将到期，不允许进入系统低功耗模式。
 - 建议：尽量采用阻塞接口，阻塞时间设置为无穷大或合理超时时间；或者采用定时器实现，并在业务认为空闲时关闭定时器。

- 投票接口使用错误
 - 举例：投票接口未成对使用。
 - 分析：系统在入睡前会检查“否决”休眠的状态，如有某个模块否决休眠，则一票即否决进入休眠模式，如果不解除对应的否决状态，系统将永远不能进入休眠模式。
 - 建议：严格检查是否成对使用，并保证否决休眠的状态持续时间尽量短，以保证尽快进入睡眠模式。

2. 为什么从深睡模式唤醒后系统执行异常？

- 举例：增加 I2C 接口的调用后，上电后系统运行正常，深睡模式唤醒后 I2C 工作异常。
- 分析：深睡模式后外设模块掉电，唤醒后需要进行重新初始化。
- 建议：使用 `uapi_pm_register_dev_ops` 接口注册 I2C 的低功耗处理函数。

3. 为什么外设对应业务概率性收发报文异常？

- 举例：SPI 设备通信发现偶尔接收不到数据，或发送数据与预期不符。
- 分析：进入深睡模式后外设均掉电，会导致接收不到对端的发送数据，或者调用完异步发送接口后，数据实际并未发送出，但系统认为没有业务正在执行或即将执行，从而进入休眠模式导致数据发送异常。
- 建议：对应业务上需要增加 `uapi_pm_add_sleep_veto`、`uapi_pm_remove_sleep_veto` 等接口调用，以通知系统是否可以进入休眠模式。

4. 为什么功耗数据会比预期偏高？

建议从硬件和软件两个方面排查。

- 硬件：检查底电流是否符合预期。
- **可选：**软件：通过以下步骤进行排查。
 - a. 通过调用维测接口 `pm_get_sleep_info` 查看统计信息中 `type` 的值，确认模式正确，查看统计量 `sleep_times`，确认曾经进入休眠模式。隔一段时间，观测其值是否持续增加，即是否在持续切换入睡唤醒状态。
 - b. 通过调用维测接口 `pm_veto_get_info` 查看是否有模块投票拒绝休眠。
 - c. 通过在注册回调 `suspend` 和 `resume` 中获取系统时钟（基于 RTC 时钟）来计算休眠时间，确认每次休眠时长是否足够，可累积计算醒睡比。

5. 业务子系统（BT、音频、显示）低功耗与系统低功耗的关系？

- 睡眠关系：子系统进行睡眠时，关闭业务侧逻辑的时钟门控或电源门控等；系统睡眠时，各个业务子系统也应处于睡眠状态。系统进入睡眠会检查业务子系统的 suspend 状态及投票状态，满足系统睡眠后，CPU 会下电，系统进入深睡模式，否则系统为浅睡模式。
- 投票关系：业务子系统睡眠时向系统取消睡眠否决票，业务子系统工作时向系统投否决票；系统睡眠前将上层业务子系统的投票结果以及其他组件注册的投票结果一起判决，只有所有投票结果都同意睡眠时系统才会进入深睡模式；否则系统进入浅睡模式。

6. 进一步降低功耗的建议

- 不使用的外设可在深睡唤醒后，关闭相应的时钟门控，建议不使用的外设在不使用时关闭，可进一步降低功耗。
- 深睡时，不使用的 IO 可配置为 GPIO 模式、配置为输入态，关闭 IO 的 IE 功能以及对上下拉配置来降低漏电流。

7. 低功耗与驱动

- 以 I2C 举例，进入深睡时，驱动相关寄存器会下电，故要保存当前寄存器状态，参考 i2c_save_reg。同理退出深睡时，驱动相关寄存器会上电继续工作，故要恢复进入深睡时的状态，参考 i2c_recovery_reg。
- 休眠唤醒时，对于 ADC 需要进行 uapi_adc_init/uapi_adc_deinit 操作，防止 ADC 采样异常。

A 唤醒源

BrandyK100 进入睡眠后，仅在唤醒源产生唤醒中断时才能将其唤醒，继续工作。不同睡眠场景下，BrandyK100 所支持的唤醒源有所差异，如下表所示，用户可根据实际业务需求选择在睡眠场景下开启或关闭某一唤醒源。BrandyK100 默认版本全部开启。

MCPU 唤醒源	浅睡场景	深睡场景
AUDIO	Yes	NO
ULP	Yes	Yes
MGPIO	Yes	Yes
M_RTC	Yes	Yes
UART_H0_RXD	Yes	Yes
UART_H1_RXD	Yes	Yes
D2M_IPC	Yes	Yes
B2M_IPC	Yes	Yes
SDIO	Yes	NO

B

GPIO 唤醒源说明

- GPIO 从芯片角度分为六组，分别是 S_EGPIO、S_HGPIO、S_MGPIO、S_AGPIO_L、S_AGPIO_R、ULP_GPIO。
 - S_AGPIO_L、S_AGPIO_R、ULP_GPIO 在芯片深睡时可以作为唤醒源来唤醒芯片，并在深睡时上述管脚的 PINMUX 功能和 PAD 功能（包括驱动能力配置、输入使能配置、上下拉配置）都能正常保持。
 - S_EGPIO、S_HGPIO、S_MGPIO 在深睡时不能作为唤醒源，并且深睡时 PINMUX 功能失效，只有 PAD 功能可以进行保持。
- 对于 S_EGPIO、S_HGPIO、S_MGPIO 几组管脚，在深睡时其 PINMUX 功能会丢失，在唤醒恢复前其 PINMUX 是作为 GPIO 的功能。对于睡眠时上下拉有强约束的管脚，需要进行特殊配置。例如对于 SPI 的 CS 管脚，需要在深睡中持续保持高，可以通过配置默认上拉来进行保证；对于 I2C 的管脚，空闲态为高，需要配置外部上拉电阻或者配置上拉。

C

低功耗优化调试说明

本节以 EVB 的单板为硬件，结合 SDK 的软件，说明功耗的基本配置。

- IO 配置原则的示例，配置文件所在的路径为“drivers/boards/brandy_evb/board_config/board_evb.h”，下面摘取部分配置进行说明：

- EVB 单板中 NOR FLASH 的管脚配置如下，其中 FLASH 的 CLOCK 和 CS 管脚只有输出，没有输入，因此配置为 HAL_PIO_IE_DISABLE，FLASH 的 DATA1 管脚中实测在下拉时深睡功耗最低，因此配置为 HAL_PIO_PULL_DOWN，其余部分采用默认配置。

```
{ HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_ENBALE }, // S_EGPIO0
{ HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
HAL_PIO_IE_ENBALE }, // S_EGPIO1
{ HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_ENBALE }, // S_EGPIO2
{ HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_ENBALE }, // S_EGPIO3
{ HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_DISABLE }, // S_EGPIO4 clk
{ HAL_PIO_FUNC_QSPI0, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_DISABLE }, // S_EGPIO5 cs
```

- 对于悬空或者不使用的管脚，统一配置为 HAL_PIO_PULL_DOWN，IE 配置为 HAL_PIO_IE_DISABLE，下述管脚即为此类。

```
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
HAL_PIO_IE_DISABLE }, // S_HGPIO0 sdio
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
HAL_PIO_IE_DISABLE }, // S_HGPIO1
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
HAL_PIO_IE_DISABLE }, // S_HGPIO2
```

```
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
HAL_PIO_IE_DISABLE },    // S_HGPIO3
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
HAL_PIO_IE_DISABLE },    // S_HGPIO4
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_DOWN,
HAL_PIO_IE_DISABLE },    // S_HGPIO5
```

- 对于部分协议的管脚，需要特殊配置，例如 UART 的管脚，协议规定空闲态管脚为高，因此在不接外设设备时管脚存在漏电，经过测试最优的配置为 UART TX 配置为 IE 的 HAL_PIO_IE_DISABLE，UART RX 配置为上拉 HAL_PIO_PULL_UP，此配置可以保证功能和功耗的最优。

```
{ HAL_PIO_FUNC_UART1, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_DISABLE },    // S_MGPIO19 uart1 tx
{ HAL_PIO_FUNC_UART1, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_UP,
HAL_PIO_IE_ENBALE },     // S_MGPIO20 uart1 rx
```

- 对于中断唤醒源管脚，不能将 IE 配置关闭，否则会导致中断无法响应，如下为 touch 的中断响应管脚配置。

```
{ HAL_PIO_FUNC_GPIO, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_ENBALE },    // S_AGPIO_R5 touch int
```

- 对于协议要求有上拉电阻的管脚，需要添加上拉电阻，此时板侧不做上下拉的配置。下述的 I2C 管脚就是此类情况。

```
{ HAL_PIO_FUNC_I2C0_M1, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_ENBALE }, // S_MGPIO11 evb2 dsp i2c smartpa
{ HAL_PIO_FUNC_I2C0_M1, HAL_PIO_DRIVE_MAX, HAL_PIO_PULL_MAX,
HAL_PIO_IE_ENBALE }, // S_MGPIO12
```

- 由于深睡时，管脚 S_EGPIO、S_HGPIO、S_MGPIO 的 PINMUX 功能无法使用，SDK 代码会默认这些管脚的 IE 使能关闭，保证深睡功耗值最优。而可以作为唤醒源的管脚 S_AGPIO_L、S_AGPIO_R、ULP_GPIO，需要客户根据实际情况在配置表中对此类管脚进行配置。

D EVB 上 IO 调试方法

IO 的调试可以直接在关闭深睡的版本上操作，通过使用“7.3 功耗 DFX”中的两个 AT 命令“AT^SETPINPULL”和“AT^GETPINPULL”，可以将所有的管脚进行遍历操作，找出管脚的最优配置。例如对于管脚 S_EGPIO1，通过查询对应的 pin 管脚号（在“drivers/chips/brandy/include/platform_core.h”文件中 enum pin_t 的枚举中定义）为 1。设置命令如下：

步骤 1 AT^SETPINPULL=1, 0 设置为不进行上下拉配置；

AT^SETPINPULL=1, 1 设置为下拉配置；

AT^SETPINPULL=1, 2 设置为上拉配置。

步骤 2 使用功耗测试仪器分别测出上述三种配置下单板的功耗值，以此来确定此管脚的最优配置。

----结束